

Research paper

Efficient Q-learning hyperparameter tuning using FOX optimization algorithm

Mahmood A. Jumaah^{a,*}, Yossra H. Ali^a, Tarik A. Rashid^b^a Department of Computer Science, University of Technology-Iraq, Al-Sina'a St., Al-Wehda District, Baghdad, 10066, Iraq^b Department of Computer Science and Engineering, University of Kurdistan Hewlêr, 30 Metre Avenue, Erbil, 44001, Iraq

ARTICLE INFO

Keywords:

FOX optimization algorithm
Hyperparameter
Optimization
Q-learning
Reinforcement learning

ABSTRACT

Reinforcement learning is a branch of artificial intelligence in which agents learn optimal actions through interactions with their environment. Hyperparameter tuning is crucial for optimizing reinforcement learning algorithms and involves the selection of parameters that can significantly impact learning performance and reward. Conventional Q-learning relies on fixed hyperparameter without tuning throughout the learning process, which is sensitive to the outcomes and can hinder optimal performance. In this paper, a new adaptive hyperparameter tuning method, called Q-learning-FOX (Q-FOX), is proposed. This method utilizes the FOX Optimizer—an optimization algorithm inspired by the hunting behaviour of red foxes—to adaptively optimize the learning rate (α) and discount factor (γ) in the Q-learning. Furthermore, a novel objective function is proposed that maximizes the average Q -values. The FOX utilizes this function to select the optimal solutions with maximum fitness, thereby enhancing the optimization process. The effectiveness of the proposed method is demonstrated through evaluations conducted on two OpenAI Gym control tasks: Cart Pole and Frozen Lake. The proposed method achieved superior cumulative reward compared to established optimization algorithms, as well as fixed and random hyperparameter tuning methods. The fixed and random methods represent the conventional Q-learning. However, the proposed Q-FOX method consistently achieved an average cumulative reward of 500 (the maximum possible) for the Cart Pole task and 0.7389 for the Frozen Lake task across 30 independent runs, demonstrating a 23.37% higher average cumulative reward than conventional Q-learning, which uses established optimization algorithms in both control tasks. Ultimately, the study demonstrates that Q-FOX is superior to tuning hyperparameters adaptively in Q-learning, outperforming established methods.

1. Introduction

Reinforcement Learning (RL) focuses on learning how to interact with the problem's environment by selecting actions that maximize reward in each state. This process resembles how children explore their environment through trial and error, learning which actions lead to desired outcomes [1]. Moreover, RL is beneficial when labelled data is unavailable, as it enables an agent to learn by interacting with the environment and receiving observations from actions [2]. RL has emerged as a powerful tool across diverse domains, proving its significance in addressing various challenges. For instance, in energy management strategies (EMS) for hybrid electric vehicles, RL enhances torque allocation by combining a heuristic rule-based local controller with a hybrid experience replay method. This integration optimizes fuel efficiency and demonstrates faster convergence than conventional techniques, even

facing challenges such as load variations and road gradients [3]. Additionally, recent advancements in supply chain management (SCM) have emphasized the integration of operations on a global scale, leading to global supply chain management (GSCM). Pontrandolfo et al. introduce an RL-based method for studying GSCM that supports integrated management and resembles autonomous agent networks (AAN). This approach is applied to networked production systems across various regions and logistics stages [4]. Furthermore, RL has been employed in environmental sustainability efforts, tackling climate change, pollution, and biodiversity decline [5]. These applications underscore the growing importance of RL in developing intelligent systems across multiple sectors.

The Q-learning is considered model-free, off-policy, and simple to implement among RL algorithms. It is well-suited for solving finite Markov decision processes (MDPs) [6]. As a model-free, Q-learning does

* Corresponding author.

E-mail addresses: cs.22.27@grad.uotechnology.edu.iq (M.A. Jumaah), yossra.h.ali@uotechnology.edu.iq (Y.H. Ali), tarik.ahmed@ukh.edu.krd (T.A. Rashid).<https://doi.org/10.1016/j.rineng.2025.104341>

Received 19 September 2024; Received in revised form 2 October 2024; Accepted 10 February 2025

not require a model of the environment and instead learns the value of actions directly from interactions with the environment. Being off-policy means the Q-learning can evaluate and improve a policy separate from the one used to generate the data. Hence, this flexibility allows the Q-learning to address various problems, including gaming, control, scheduling, and security [7,8]. However, the Q-learning addresses the issue by approximating an action-value function, which facilitates the selection of optimal actions for each state in the problem [9]. Additionally, the process of the Q-learning involves running multiple episodes to iteratively refine the policy and maximize reward [10,11].

However, despite the potential of RL algorithms to make reliable decisions and achieve maximum reward, they often require manual hyperparameter (HP) tuning. These HP, which are pre-initialized parameters, significantly influence the learning process [12]. Consequently, the challenge lies in identifying the most effective HP values, as even minor adjustments can impact problem-solving significantly. Moreover, the Q-learning does not update these HP during the learning process; they remain fixed across all steps. Manual tuning is complex and time-consuming, and although various tuning methods—such as random adjustments—are available, these approaches may not be universally practical due to varying environmental conditions [13]. Consequently, it is crucial to employ adaptive HP tuning in Q-learning, as using fixed HP values can result in suboptimal learning outcomes, especially in dynamic or complex environments [14]. The effectiveness of Q-learning depends on appropriately setting parameters such as the learning rate (α) and discount factor (γ), which influence the agent's ability to explore and exploit efficiently. Without adaptive tuning, the algorithm may struggle to find optimal policies, eventually limiting its effectiveness in solving real-world control tasks [12]. Therefore, an adaptive tuning method is necessary to enhance the algorithm's performance.

Optimization involves determining the best solution to a problem by maximizing or minimizing an objective function, which requires adjusting parameters to achieve the desired outcome. Various optimization algorithms have recently gained prominence in machine learning, notably gradient-based optimization (GBO) and golden eagle optimization (GEO), which are effective for solving complex optimization challenges. GBO, rooted in Newton's and gradient numerical methods, utilizes two primary procedures: the gradient search criterion (GSR) and the locally extracting operator. The GSR enhances the search efficiency and accelerates convergence rates through a gradient-based technique [15]. In contrast, GEO is inspired by the intelligence of golden eagles, particularly their hunting behaviour, which includes a cruising phase followed by aggressive tactics. This unique inspiration contributes to GEO's effectiveness across various applications [16]. Another noteworthy algorithm is sand cat swarm optimization (SCSO), which draws from the adaptations of sand cats, such as their fur-covered paws and ability to detect low-frequency sounds, allowing them to thrive in harsh desert environments. SCSO enhances problem-solving capabilities by quickly identifying near-optimal solutions, like sand cats locating their prey. The algorithm begins by initializing a population of search agents within a defined space, where a fitness function evaluates each iteration, guiding the search towards the intended objective—whether it involves minimization or maximization [17]. Additionally, the shrike optimization algorithm (SHOA) takes inspiration from the behaviour of shrike birds. These birds migrate to survive and exhibit specific breeding and territory management adaptations. SHOA has demonstrated its superiority in benchmark tests, effectively addressing multi-modal problems compared to established algorithms [18]. Another innovative approach is child drawing development optimization (CDDO), which is inspired by cognitive development and children's learning behaviours, particularly the golden ratio's role in optimizing the aesthetics of their art. The golden ratio was first introduced by the mathematician Fibonacci [19]. CDDO mimics cognitive learning stages, progressing from scribbling to advanced patterns by adjusting hand pressure, width, and length parameters. This optimization enhances artistic results and supports children's intelligence development and achievement of shared goals. Notably,

CDDO excels in exploring new solutions and effectively avoids local minima by thoroughly covering promising regions and exploiting the best solutions in the design space [20]. Moreover, the ant nesting algorithm (ANA), inspired by *Leptothorax* ants, simulates their behaviour in seeking optimal locations for grain placement while constructing nests. ANA is a continuous algorithm that updates search agent positions based on a rate of change, uniquely calculating this rate using past and current solutions. It generates weights via the Pythagorean theorem to guide agents in exploration and exploitation. As a result, ANA outperforms leading metaheuristic algorithms in various test cases, delivering highly competitive results [21]. The fitness dependent optimizer (FDO) is a PSO-based swarm intelligence algorithm that draws from the intriguing behaviour of bee swarming and collective decision-making. Unlike traditional bee algorithms, FDO calculates velocity using the fitness function value to generate weights that effectively guide search agents in both exploration and exploitation. FDO has consistently outperformed several well-known algorithms in various tests, showcasing impressive improvements and stability [22].

Despite these advancements, existing HP tuning methods that use optimization algorithms, such as [23–26], often need more adaptability, failing to adapt to the changing conditions of different environments. Fixed HP can lead to suboptimal performance, particularly in complex tasks where conditions may change unexpectedly. This gap emphasizes the need for more adaptive approaches in RL. This study addresses these shortcomings by suggesting a new method for adaptive HP tuning. This approach enhances Q-learning's performance and contributes to the broader application of RL across diverse domains. However, this paper introduces the Q-learning-FOX (Q-FOX), which utilizes the Fox optimizer (FOX) for HP tuning in the Q-learning. Inspired by the natural hunting behaviour of red foxes, the FOX suggests an adaptive and efficient solution to the limitations of HP tuning in the Q-learning algorithm. The FOX, introduced in 2023 by Mohammed and Rashid [27], is inspired by the behaviour of red foxes in the wild, such as walking, jumping, searching for food, and hunting. These actions were modelled using mathematical equations, allowing FOX to outperform many established optimization algorithms. FOX has been used to solve various benchmark test functions and many engineering design problems [28,29].

Moreover, a new objective function is proposed to accommodate the single-objective nature of FOX, integrating the average Q -values and learning error while prioritizing maximizing Q -values over minimizing learning error to ensure stability in the learning process. Furthermore, to evaluate the performance of the Q-FOX, experiments were conducted using two control tasks from OpenAI Gym environments: Frozen Lake and Cart Pole. Frozen Lake involves navigating a slippery grid world to reach a goal while avoiding hazards, whereas a Cart Pole requires balancing a pole on a moving cart to maintain stability. These tasks serve as benchmarks for RL algorithms, testing the robustness of the proposed method in diverse control situations [30]. Finally, experimental results show that the proposed Q-FOX improves Q-learning performance by adapting to optimize the learning rate (α) and discount factor (γ). Furthermore, the proposed method shows an increased cumulative reward through the adaptive HP tuning, improving Q-learning performance. Therefore, the main contributions of this paper are:

1. Proposing the Q-learning-FOX (Q-FOX), a new method that integrates the FOX for adaptive HP tuning in Q-learning.
2. A novel objective function is proposed that combines average Q -values and learning error, prioritizing the maximization of Q -values to ensure stability in the learning process.
3. The Q-FOX method significantly improves Q-learning performance, evidenced by increased cumulative rewards during experiments on benchmark control tasks.
4. The study addresses the challenge of manual hyperparameter tuning by providing an adaptive solution which allows for dynamic adjustments of HP (α , γ) in Q-learning.

The structure of this paper is organized as follows: Section 2 reviews recent studies relevant to the problem of HP tuning. Section 3 details the proposed Q-FOX, including the background of the FOX, Q-learning, and OpenAI Gym control tasks. Furthermore, Sections 4 and 5 list and discuss the experimental results, while Section 6 concludes the study, review the limitations, and suggests future work directions.

2. Literature review

Recent advancements in HP tuning for RL algorithms have attracted considerable attention from researchers. This section reviews critical studies utilizing various optimization algorithms across different applications and applying Q-learning to control tasks such as Frozen Lake and Cart Pole. These approaches will be compared to the proposed method employed in this research, highlighting contributions and existing gaps that this work aims to address.

A Genetic Algorithm (GA) is an evolutionary optimization algorithm that models a natural selection approach by iteratively improving a population of candidate solutions. GA explores the solution space using selection, crossover, and mutation operators. The fitness of each solution $f(x)$ is evaluated, and the best individuals are selected to create the next generation, enabling the algorithm to converge towards optimal solutions [31]. Further, Fernandez and Caarls utilized a GA for HP tuning in Q-learning. Their method, tested on the Pendulum control task, demonstrated notable improvements in maximizing reward and accelerating learning speed. Despite these successes, they identified a critical limitation: the GA's propensity to get trapped in local minima, which necessitated multiple restarts to achieve optimal performance. This insight underscores the challenges of ensuring robust convergence in complex HP optimization tasks [23]. Likewise, the grey wolf optimizer (GWO) is a nature-inspired algorithm based on the social hierarchy and hunting behaviour of grey wolves in the wild. It mimics the leadership structure of wolves, using alpha, beta, delta, and omega wolves to guide the search for optimal solutions [32]. Accordingly, Tapia et al. introduce a novel hyperheuristic binarization framework to balance exploration and exploitation in solving combinatorial optimization problems. The framework integrates Q-learning, a reinforcement learning algorithm, with a GWO at the lower level to handle binary issues, such as the set covering problem (SCP). The upper level uses ant colony optimization (ACO) to guide the selection of the best binarization strategy. By leveraging Q-learning, the proposed approach dynamically adjusts the binarization scheme based on real-time feedback from the solution space, allowing for an enhanced trade-off between exploration and exploitation. The study highlights the effectiveness of this method in improving solution quality and convergence, demonstrating its advantage over traditional optimization algorithms [24].

In a related study, Fajri et al. present a hybrid model combining the Bees algorithm (BA) and Q-learning to enhance feature selection in the diagnosis of coronary heart disease. BA is a nature-inspired algorithm that mimics the foraging behaviour of bees to identify optimal solutions [33]. This methodology integrates Q-learning with BA to make the search process adaptive and improve feature selection efficiency. The approach was tested on multiple datasets, demonstrating improved classification accuracy compared to traditional algorithms. However, the study identifies the need for further optimization of execution time with larger datasets [25]. Correspondingly, particle swarm optimization (PSO) is a nature-inspired optimization algorithm that mimics the social behaviour of birds. In PSO, a population of candidate solutions (particles) moves through the search space to find the optimal solution. Each particle adjusts its position based on its own experience and the neighbouring particles' experience, using velocities and positions updated based on the best positions. The objective is to explore the solution space and converge towards the global best solution [34]. A recent study suggested a new algorithm that combines PSO and Q-learning called AQLPSO. Each particle in this algorithm has four states and actions. Based on the accumulated experience from Q-learning, the particles can

select the appropriate action in different states. The simulation results showed that the AQLPSO algorithm exceeded the performance of the PSO, PSO-SA, and RMPPO algorithms. It successfully addressed the UAV path planning challenge in complex 3D flight environments by minimizing the risk of local optima, enhancing efficiency, and accelerating convergence toward the global optimal solution [26].

Otoni et al. introduced a new method for addressing sequential ordering problems (SOPs) using RL algorithms. Their approach hybridizes Scott-Knott statistical methods with HP tuning in SARSA and Q-learning. By testing their method against benchmarks from the TSPLIB library, they indicated that the SARSA algorithm outperforms Q-learning in solving SOPs. This finding highlights SARSA's superior performance in effectively managing the complexities of SOPs compared to Q-learning [35]. Along with this, a research team from DeepMind presented a study focusing on HP selection in offline RL. They examined methods for selecting the best policy from multiple learned policies using varying HP. Their extensive evaluations showed that offline RL algorithms could be more robust to HP variations, and the methods for approximating Q-values significantly impact the overall HP tuning. However, they concluded that practical HP tuning is achievable even in complex tasks involving high-dimensional observations and action spaces [36].

Many recent studies focus on solving control tasks using RL algorithms, including the original or improved Q-learning. Gupta et al. demonstrated a comparison study on the Frozen Lake control task between statistical Q-learning and cognitive instance-based learning (IBL) algorithms. The Q-learning and IBL agents were tested on different grid sizes of Frozen Lake. The IBL algorithm outperformed Q-learning by requiring less learning time, generalizing better, and scaling effectively to larger environments [37]. However, the study did not address Q-learning's limitations, underscoring the challenge of developing an enhanced version capable of effectively solving Frozen Lake and other control tasks. Additionally, Mak et al. propose the Convergence-Aware Sampling with Screening (CAESAR) aggregation scheme. CAESAR combines convergence-aware sampling with a screening mechanism to selectively integrate knowledge from agents learning in similar MDPs, improving overall learning efficiency. The empirical results, tested in a custom gridworld environment and the Frozen Lake control task, validate CAESAR's effectiveness in enhancing agent learning across varied environments [38]. On the Other hand, Mishra and Arora proposed two Q-learning-based approaches for solving the Cart Pole control task. They used mean squared error (MSE) and mean absolute error (MAE) as reward functions. The evaluation focused on how well each approach balanced the pole, intending to maximize the reward. Their findings indicate that Q-learning with MAE tends to ignore outliers while Q-learning with MSE gives more weight to outlier values [39].

Recent advancements in RL algorithms have led to improved tasks such as reward maximization and exploration-exploitation balance. However, existing HP tuning methods like PSO, GWO, GA, and BA often face challenges in escaping local minima across a wide range of control tasks. These methods typically rely on fixed or semi-adaptive tuning strategies, limiting their effectiveness in complex or evolving environments. In contrast, the proposed Q-FOX introduces an adaptive hyperparameter tuning mechanism that dynamically adjusts learning parameters based on the environment's feedback, addressing the limitations of existing algorithms. Furthermore, the proposed objective function has tackled the challenges of local minima. This adaptive capability allows Q-FOX to achieve more stable and efficient learning, particularly in tasks where other algorithms struggle with convergence and adaptability. Refer to Table 1 for a summary of the literature review.

3. Methodology

An adaptive HP tuning method for Q-learning has been proposed, utilizing the FOX instead of traditional tuning methods like manual or random. The proposed method aims to optimize the HP adaptively to maximize reward, thereby improving the performance of Q-learning.

Table 1
Summary of Methodologies and Key Findings of the literature review.

Methodology	Key Findings	Ref.
GA	Improves reward and learning speed in Q-learning but struggles with local minima.	[23]
GWO	Combines Q-learning and GWO for enhanced exploration-exploitation balance in binary problems.	[24]
BA	Integrates Q-learning for improved feature selection in coronary heart disease diagnosis.	[25]
PSO	AQLPSO combines PSO and Q-learning, outperforming PSO variants in UAV path planning.	[26]
Hybrid RL	SARSA outperforms Q-learning in sequential ordering problems.	[35]
Offline RL	Effective HP tuning is possible even in complex tasks with high-dimensional spaces.	[36]
Q-learning and IBL	IBL outperform Q-learning in the Frozen Lake control task by requiring less training and better generalization.	[37]
CAESAR	Enhances learning efficiency across varied environments.	[38]
Q-learning with MAE/MSE	MAE ignores outliers, and MSE gives more weight to outliers in the Cart Pole control task.	[39]

The following sections will explain the algorithms involved, including Q-learning, FOX, and the proposed Q-FOX.

3.1. Q-learning algorithm

RL is one of the most promising fields in artificial intelligence, which allows an agent to learn through interactions with its environment. Unlike supervised learning, RL does not rely on training data or target labels; instead, the agent learns by taking actions within the environment and striving to maximize reward [40,41]. This learning process is analogous to how humans learn to play a video game: the agent continuously attempts to improve performance by avoiding actions that led to failure and repeating those that resulted in success [42]. A notable advancement in RL is Q-learning, introduced by Watkins in 1989, which is considered a seminal breakthrough in the field. Q-learning is defined by the Equation (1).

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right] \quad (1)$$

Where $Q(s_t, a_t)$ is the action-value function, representing the cumulative reward expected if action a is taken in the state s , action-value pairs are placed in a table called Q -table, which is an array that has dimensions of (No. of states, No. of actions). s and a represent the current state and action, respectively. The agent will obtain the reward r and move to another state s_{t+1} by taking action a in the current state s . The α and γ are learning rate (alpha) and discount factor (gamma), respectively [10].

Q-learning is designed to identify the best sequence of actions that follow a policy in an MDP by determining the optimal action-value function using the update rule from Equation (1). The agent interacts with the environment to observe reward and progressively updates the action-value function to improve its policy estimates [43]. This interaction occurs iteratively, allowing the algorithm to adjust its policy based on observations of the environment. After the learning phase, the agent should be capable of selecting the optimal action (policy) for a given state using the estimated Q -table, as described by Equation (2).

$$\pi^*(s) = \arg \max_a Q(s, a) \quad (2)$$

Where π^* represents the optimal policy, and $\arg \max_a$ is the function that returns the action a that maximizes the Q -values based on the learned policy [44,45].

Q-learning selects actions based on the epsilon greedy method. With probability $1 - \epsilon$, the agent chooses the action with the highest Q -value, exploiting its current knowledge to maximize expected reward. Conversely, with probability ϵ , the agent selects a random action to encourage the exploration of new possibilities. Hence, this strategy effectively

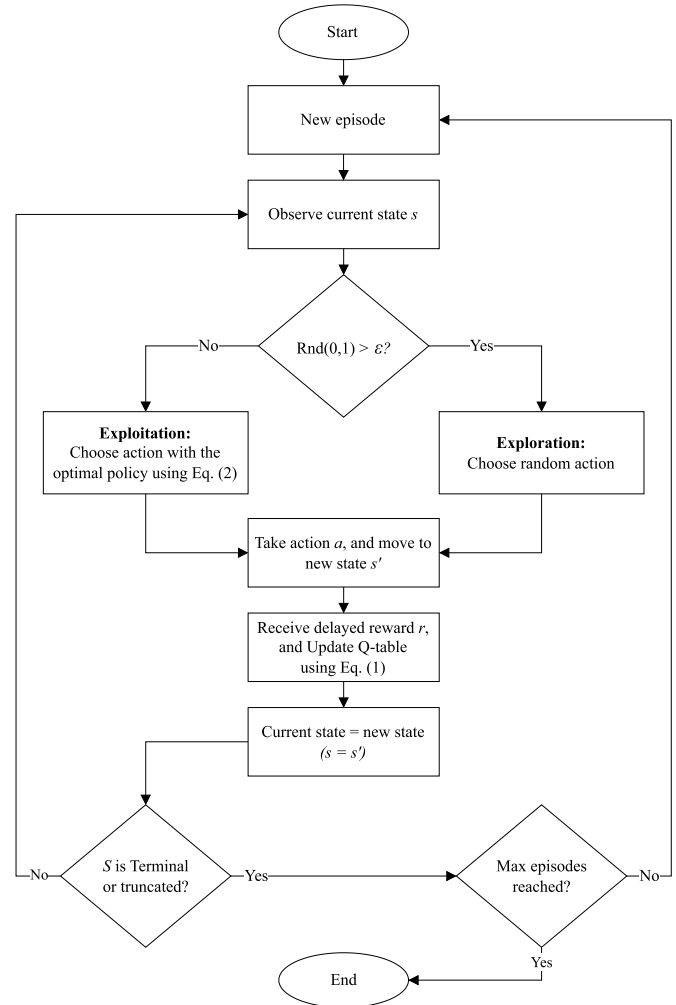


Fig. 1. Flowchart of Q-learning [46].

balances the need to exploit known optimal actions while learning potentially better alternatives. Fig. 1 provides a visual description of the Q-learning [46].

However, the Q-learning involves several HP that significantly impact the agent's performance and behaviour, as detailed in Table 2. Critical HP includes the learning rate (α), discount factor (γ), and epsilon (ϵ), all of which must be initialized before the learning phase begins.

Table 2
Summary of hyperparameter in the Q-learning [54].

HP	Purpose	Range
α	The extent to which new information overrides old information.	[0, 1]
γ	Importance of future reward in the decision-making process.	[0, 1]
ϵ	Impact the trade-off between exploitation and exploration.	[0, 1]

The learning rate (α) is crucial for updating the action-value function at each time step. It dictates how much new information should override old information, with its value typically ranging from [0, 1] [47,48]. A lower α makes the algorithm less responsive to new information, resulting in slower convergence, while a higher α accelerates updates but can cause instability. Thus, selecting an appropriate α is essential for optimal performance [49].

The discount factor (γ) influences how future reward are considered in the agent's decision-making process, with values ranging from [0, 1] [50]. A higher γ places more emphasis on future reward, whereas a lower γ prioritizes immediate reward. This parameter balances the trade-off between immediate and future reward, affecting the final policy [51].

The epsilon (ϵ) parameter manages the exploration-exploitation trade-off [52]. From [0, 1], a higher ϵ favours exploitation, while a lower value promotes exploration. This parameter is crucial for determining the ratio between exploring new and exploiting known actions, directly impacting the agent's action-selection process [53].

While the fixed values for HP like α and γ offer a straightforward implementation, they can restrict the agent's learning efficiency, especially in complex or dynamic environments [13]. These fixed settings often result in suboptimal performance, either slowing convergence or not reaching the maximum possible rewards [55]. Adaptive hyperparameter tuning addresses this issue by allowing these parameters to change dynamically based on the agent's interaction with the environment. As a result, the Q-learning agent can achieve more stable and effective learning outcomes.

3.2. Fox optimizer algorithm

FOX is a novel optimization algorithm inspired by the hunting behaviour of red foxes. The selection of FOX is based on its advanced mechanisms that address the limitations of existing swarm-based algorithms, achieving a better balance between the exploration and exploitation phases. Unlike PSO, GWO, and BOA, FOX's random walk and distance measurements enable a more refined search process. Its jump mechanism enhances the ability to escape local optima, resulting in superior performance on benchmark test functions and real-world engineering design problems [27]. This statistically validated performance highlights FOX's robustness, adaptability, and applicability to complex optimization challenges, including reinforcement learning, where practical exploration and exploitation are crucial. However, FOX aims to identify the optimal solution by evaluating the best fitness values across a population of search agents. The FOX operates through multiple iterations with numerous agents (i.e. 30), each seeking to find the optimal fitness value (better solution) [27]. FOX consists of two primary phases: exploration and exploitation. During the exploration phase, agents employ a random walk strategy to locate potential solutions, analogous to how red foxes search for prey. They use their ability to detect ultrasound signals to assist in this search [27]. Upon detecting these signals, agents transition to the exploitation phase. In this phase, the agents estimate the distance to their prey based on the time it takes for the ultrasound signal to reach them, given that sound travels at a constant speed of 343 meters per second [56]. Consequently, FOX employs a distinct method for measurement, where agents jump when they estimate they can capture the prey based on the elapsed time of the sound signal [27,57,58]. Consequently, the agent's success in capturing prey is closely tied to its ability to interpret the sound's travel time while jumping accurately.

$$DSF_i = s \times T_i \quad (3)$$

The sound distance from the FOX agent (FA) is represented by the vector DSF as given in Equation (3). Concurrently, the sound speed in the medium is defined by the variable s , calculated using Equation (4). The vector T represents arbitrary numbers from 0 to 1. The variable i denotes the current iteration of the process [27,59].

$$s = BestPosition_i \times T_i \quad (4)$$

The distance of FA from its prey (DFP) is defined as follows [27]:

$$DFP_i = DSF_i \times 0.5 \quad (5)$$

Once the FA determines the distance to its prey, it must immediately jump to catch it. The required jump height is computed using the following equation [27]:

$$jump_i = 0.5 \times 9.81 \times t^2 \quad (6)$$

where 9.81 represents the acceleration due to gravity, and t is the average time taken by the sound to travel, computed by using the vector T from Equation (3) [27,60].

The FA moves to a new position during the exploration and exploitation stages. In the exploitation stage, the new position is determined using one of the following equations [27,61]:

$$X_{i+1} = DFP_i \times jump_i \times c_1 \quad (7)$$

$$X_{i+1} = DFP_i \times jump_i \times c_2 \quad (8)$$

The values of c_1 and c_2 are 0.180 and 0.820, respectively, determined based on the jump dynamics of the FA. The agent's jumps are directed either towards the northeast or the opposite direction. For the exploration stage, the new position is calculated using the following equation [27,62]:

$$X_{i+1} = BestX_i \times rand(1, dim) \times min(tt) \times a \quad (9)$$

Here, tt is the time average, equal to the sum of the vector T divided by the dimension of the problem (dim). The variable a is calculated as $2 \times (i - [1/\max(i)])$.

The FOX, inspired by the hunting behaviour of red foxes, employs a static trade-off strategy, equally balancing the exploration and exploitation stages at 0.5 each. In the exploration phase, the optimizer mimics the fox's prey detection capabilities through random walks. This approach simulates how a fox might search for prey in its environment, allowing the optimizer to explore a diverse set of potential solutions [27]. Conversely, the optimizer focuses on refining its search during the exploitation phase. Moreover, it calculates the distance to the prey, determines the required jump height, and updates the agent's position based on these calculations. This phase aims to converge on the optimal solution by exploiting the most promising areas identified during the exploration phase [27]. To better understand the operational flow and processes, see Fig. 2, which visually represents the FOX.

3.3. Q-learning-FOX (Q-FOX)

This study proposes a new hybrid RL method, Q-FOX, created by integrating FOX with Q-learning to enhance learning efficiency and performance. The Q-FOX improves the problem-solving process by providing adaptive HP tuning, replacing the constant HP, and eliminating the need for human intervention in Q-learning as visualized in Fig. 3 and Algorithm 1. The FOX must first be initialized by specifying the number of iterations (i), FAs, and the solution bounds (lower bound and upper bound), which are exhaustively detailed in FOX (Section 3.2). The α bounds are set to [0.01, 0.1] and γ bounds are set to [0.5, 0.99]. However, the source code of the Q-FOX method is publicly available on GitHub at [63].

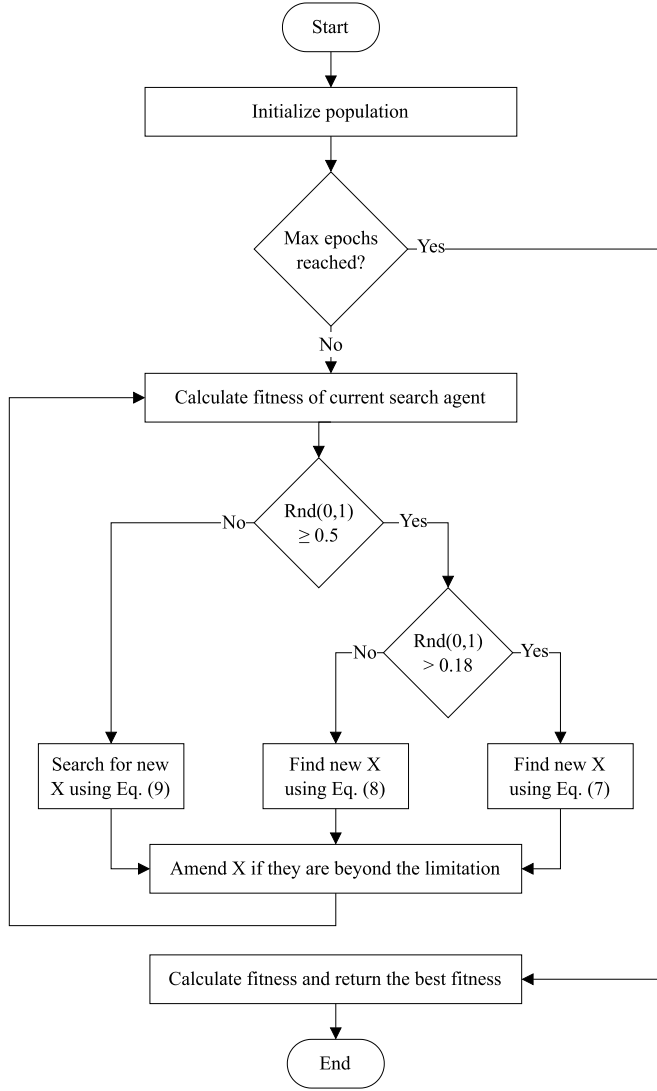


Fig. 2. Fox optimizer algorithm flowchart [27].

Algorithm 1 Q-learning-FOX (Q-FOX).

```

1: Initialize  $Q(s, a)$  arbitrarily (initial  $Q$ -values for all state-action pairs)
2: loop for each episode
3:   Initialize  $s$  (start state)
4:   repeat for each step of the episode
5:     Choose action  $a$  from state  $s$  using policy derived from  $Q$  (e.g.,  $\epsilon$ -greedy)
6:     Take action  $a$ , observe reward  $r$  and new state  $s'$ 
7:     Compute  $q'$  based on optimized  $(\alpha, \gamma)$  using FOX  $\triangleright$  Refer to Eq. (1)
8:     Compute  $q$  based on typical  $(\alpha, \gamma)$   $\triangleright$  Refer to Eq. (1)
9:      $Q(s, a) \leftarrow \max(q', q)$   $\triangleright$  Update  $Q$ -Table
10:     $s \leftarrow s'$   $\triangleright$  Move to the next state
11:   until  $s$  is terminal
12: end loop
  
```

$$\text{if } \epsilon > \epsilon_{\min} : \quad \epsilon \leftarrow \epsilon \times \epsilon_{\text{decay}} \quad (10)$$

$$\epsilon_{\text{decay}} = \left(\frac{\epsilon_{\min}}{\epsilon} \right)^{\frac{1}{N}} \quad (11)$$

The Equation (10) is used to update the ϵ value iteratively at each episode, ensuring that the exploration rate decreases over time. This dynamic adjustment improves upon the static ϵ -greedy method by dynamically allowing for a more gradual transition from exploration to exploitation. Where the initial ϵ is set to 1, and the $\epsilon_{\min}$ represents the

minimum value of ϵ can be reached (i.e. 0.01) to prevent ϵ decreasing further below this threshold.

Moreover, during the Q-learning process, the FOX provides the current solution, representing the HP (α, γ) used for updating Q -values in the Q-learning. At each episode, the following steps are performed:

1. **Action Selection:** An action a is chosen from the current state s using a policy derived from the current Q -values, such as the ϵ -greedy method.
2. **Optimization Process:** After taking action a , the resulting observation (r, s') is passed to the FOX. The FOX evaluates the current HP (α, γ) and optimizes them based on their performance in the environment. This is accomplished by leveraging the reward from the environment, which is expressed through the proposed objective function in Equation (12).
3. **Dynamic Adjustment:** The FOX dynamically adjusts these HP based on the computed fitness value, which integrates average Q -values and learning error. This adaptation allows Q-FOX to respond to the changing dynamics of the environment, thus enhancing learning efficiency.
4. **Q -value Update:** Following the optimization, the method compares the optimized Q -value (q') calculated using the optimized HP with the existing Q -value (q). If the optimized value q' is greater than the current q , the algorithm updates the Q -table with this new value. Otherwise, it retains the existing Q -value, ensuring continuity in the learning process.

This process is repeated iteratively, guided by the fitness value of the objective function to find the optimal HP. This update process is sensitive to the objective function, highlighting the need for a well-designed function to manage this sensitivity effectively. Hence, a novel objective function has been proposed that integrates both average Q -values and learning error into a unified objective function as detailed in Equation (12). The function prioritizes the averaged Q -values over the learning error, ensuring that the HP is optimized to maximize the reward, find the optimal policy, and improve the overall learning performance.

The proposed objective function, as defined in Equation (12), aims to balance two critical components: maximizing the average Q -values (aq) and minimizing the learning error (e). The integration of these components is expressed as follows:

$$f_{\max} = aq \times w_1 - e \times w_2 \quad (12)$$

$$e = \frac{1}{n} \sum_{i=1}^n \left(Q(s, a) - r + \gamma \max_{a'} Q(s', a') \right)^2 \quad (13)$$

Where n is number of samples (or updates) in the Q -table, w_1 and w_2 is the weight scalar that prioritizes the Q -values (i.e., $w_1 = 0.7$ and $w_2 = 0.3$). The weights prioritize maximizing Q -values while addressing the importance of learning accuracy through error minimization. A higher weight is given to the average Q -values to guarantee that the primary objective remains to achieve higher expected returns from the policy, thus guiding the learning agent toward more rewarding actions.

The learning error e , calculated using MSE in Equation (13), quantifies the deviation of the Q -values from the expected rewards. It is crucial to minimize this error, as excessive learning errors can lead to instability in the Q -learning process [64]. The priority shifts towards maximizing Q -values, ensuring that the agent is learning accurately and exploring optimally. Additionally, proposed objective function is used with the second half of episodes to ensure the stability of the learning process. Specifically, the computation starts from $i = \text{episodes} - \text{episodes}/2$. For instance, with a total of 500 episodes, the fitness function is computed from episode 250 (start) to episode 500 (end).

The Q-FOX initiates when the Q-learning enters the Q-learning's iterative steps. During each step, an action a is first taken in the current state s to observe the environment. Subsequently, the observations are fed

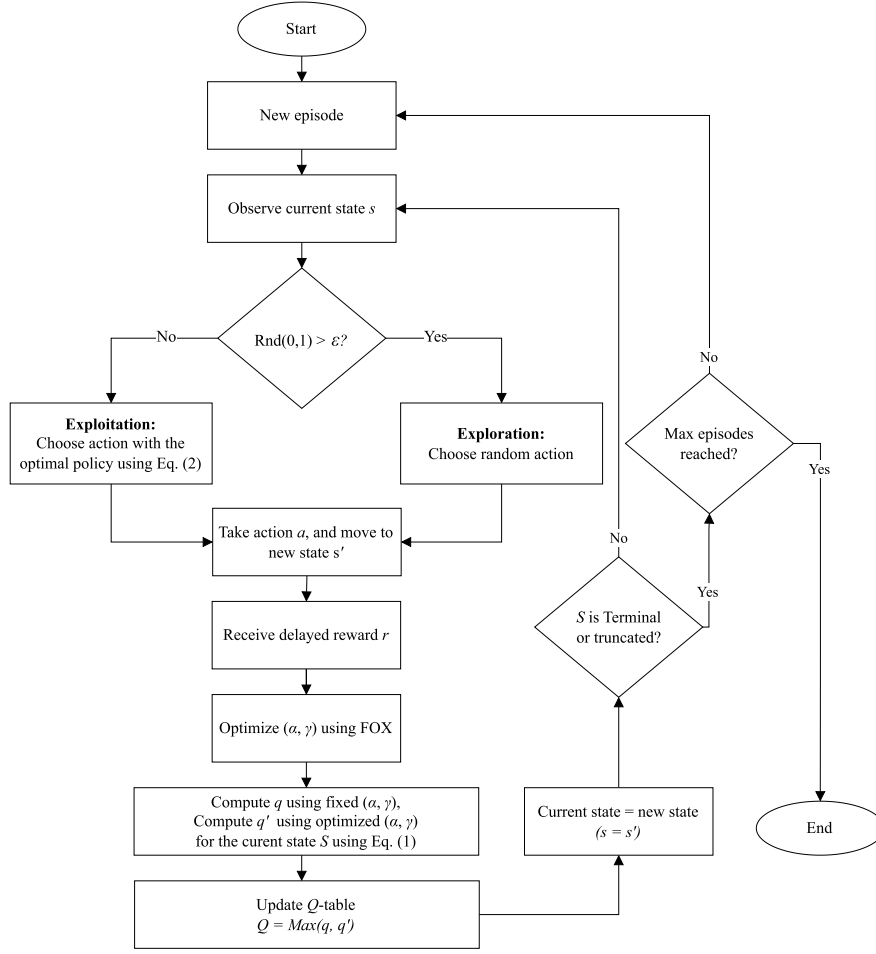


Fig. 3. The proposed Q-learning-FOX flowchart (Q-FOX).

into the FOX. The FOX then works to optimize the HP (α, γ) to maximize the average Q -values. This process aims to identify the best solutions that optimize the objective function, as described in Equation (12). Once the optimization process for the current step is completed, the method compares the optimized Q -value (q') calculated using the optimized (α, γ) with the typical q -value (q) obtained using the typical (fixed) (α, γ) . If q' is greater, the method incorporates this value to update the Q -values (Q -table). Conversely, if no improvement is observed, the learning process continues to the next step with typical q , which is repeated until a maximum number of episodes is reached.

3.4. Control tasks

Standard control tasks operate as fundamental benchmarks for evaluating the performance of RL algorithms. This study employed two well-known control task environments—Frozen Lake and Cart Pole—sourced from OpenAI's Gym environments [30], which are explained in the following paragraphs.

Frozen Lake. In the Frozen Lake environment, an agent must navigate a grid world on a frozen lake, which introduces obstacles and adds complexity to the RL problem [30]. The objective is to move from the Start (S) to the Goal (G) while avoiding Holes (H), see Fig. 4a. The action space is discrete, consisting of four actions: LEFT (0), DOWN (1), RIGHT (2), and UP (3). The observation space, also discrete, represents the agent's position as $current_row \times n_{rows} + current_col$. The reward are +1 for reaching the G and 0 for falling into an H or stepping on a frozen tile (F) [65].

Cart Pole. The Cart Pole environment is modelled after the problem described by Barto, Sutton, and Anderson [66]. A pole is attached to a cart moving along a frictionless track in this environment. The objective is to apply forces F to the cart, pushing it either left (F_0) or right (F_1) to balance the pole perpendicular, see Fig. 4b. The observation space is represented by a vector $X \in \mathbb{R}^4$, comprising the cart position $x \in [-4.8, 4.8]$, cart velocity $\dot{x} \in (-\infty, \infty)$, pole angle $\theta \in [-0.418, 0.418]$ radians, and pole angular velocity $\dot{\theta} \in (-\infty, \infty)$. However, the episode terminates if θ exceeds $\pm 12^\circ$ or x exceeds ± 2.4 , with a reward of +1 awarded for each time step that the pole remains balanced [66,67]. Moreover, since the Cart Pole states are continuous, they are discretized for use with the Q-learning. A Q -table is created with a dimension of $1 \times 1 \times 6 \times 3 \times 2$ to store reward for all discretized state-action pairs. The episode ends if the maximum length of 500 steps is reached. Thus, for an effectively learned algorithm, the cumulative reward should approach the maximum possible value of 500.

3.4.1. Procedure for applying Q-FOX to Frozen Lake and Cart Pole control tasks

The following procedure summarizes the steps in applying the Q-FOX algorithm to control tasks using the OpenAI Gym library. This study will focus on two specific tasks: 'FrozenLake-v1' and 'CartPole-v1.' Using the FOX, the Q-FOX approach optimizes the Q-learning process by adaptively tuning the HP (α, γ) . This adaptive method improves the performance of the learning agent by combining RL with optimization algorithms.

1. Initialization:

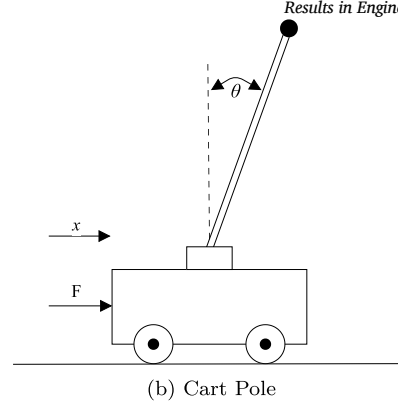
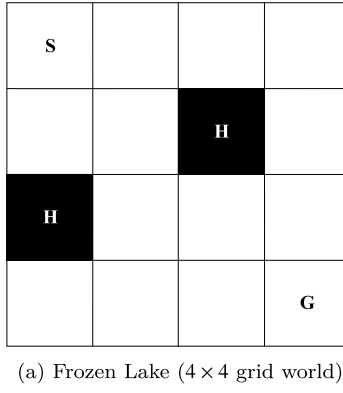


Fig. 4. Control task problems, adapted from [68,69].

- Initialize the environment using OpenAI Gym library: `env = gym.make('CartPole-v1')` or `env = gym.make('FrozenLake-v1')`. You can use other OpenAI Gym in the same way.
 - Initialize the Q -table with zeros for all state-action pairs.
 - Set the initial values for the learning rate (α), discount factor (γ), exploration rate (ϵ), and the number of episodes.
2. **Optimizer Setup:**
- Initialize the FOX with a specified population size and number of iterations.
 - Define the objective function:
- $$f(\alpha, \gamma) = aq \times 0.7 - e \times 0.3$$
- where aq represents the average Q -values and e the learning error. Refer to Equations (12) and (13).
- Set the problem's bounds: $\alpha \in [0.01, 0.5]$ and $\gamma \in [0.5, 0.99]$.
3. **Learning Loop:** For each episode:
- (a) Reset the environment and obtain the initial state, $s = \text{env.reset}()$.
 - (b) For each step within the episode:
 - Choose an action a using an ϵ -greedy method based on the current Q -table.
 - Perform the action a and observe the environment. $r, s' = \text{env.step}(a)$.
 - Apply optimization using FOX:
 - Generate a new set of solutions (candidate values for α and γ).
 - Evaluate each solution based on the objective function $f(\alpha, \gamma)$.
 - Select and return the optimal values of α and γ that maximize the objective function.
 - Update the Q -values:
 - Compute q using the typical (fixed) (α, γ) and q' using the optimized (α, γ) as per Equation (1).
 - Update the Q -table:
- $$Q(s, a) \leftarrow \max(q, q')$$
- Update the state $S \leftarrow S'$.
 - (c) **Epsilon Decay** Gradually reduce ϵ to promote exploitation over exploration as learning progresses. Refer to Equation (10).

4. **Termination:** The learning process concludes after the predefined number of episodes, and the final Q -table represents the learned optimal policy.

Following this structured procedure, Q-FOX effectively adapts the HP during learning for both 'FrozenLake-v1' and 'CartPole-v1' tasks, optimizing the Q-learning agent's performance. The FOX ensures that the selected learning rate (α) and discount factor (γ) are continually fine-tuned based on an objective function that prioritizes the average

Q -values. This objective function is crucial because it directly influences the learned policy's ability to gain more rewards, guiding the agent toward optimal decision-making. By evaluating candidate solutions using this fitness function, which combines the Q -values and MSE, see Equation (13), the FOX effectively updates its population, ensuring that only the significant solutions are selected. This approach enables the agent to learn optimal policies more efficiently, achieving improved performance in the given control tasks.

4. Experimental results

The performance of several optimization algorithms, including PSO, GA, GWO, and BA, as well as fixed and random tuning methods, was evaluated and compared against the proposed Q-FOX. The experimental setup involved the evaluations under consistent settings to standardize the results. Furthermore, the experiments were conducted in a controlled environment using hardware and software specifications. The operating system was Ubuntu 24.04.1 LTS, running on an MSI, Ltd. GL63 8RD PC equipped with an Intel® Core™ i7-8750H processor (12 threads) and 32 GiB of memory. The experiments were implemented in Python 3.12.3, utilizing the gymnasium library version 0.29.1 for the Q-learning simulations (control tasks). The proposed method was tested over 500 episodes to ensure a thorough assessment of its performance. The optimization algorithms were executed for 50 epochs and a population of 20 agents, allowing for practical solution space exploration without excessive computational demands. However, the choice of 500 episodes was determined through trial and error. This number was sufficient for most tasks, allowing effective convergence while balancing exploration and exploitation. For the more complex tasks like 7×7 Frozen Lake control task grid size, the number of episodes has been extended to 2000, ensuring sufficient learning. Additionally, various methods for determining the suitable number of episodes are discussed in [70].

Moreover, the fixed and random methods represent the conventional Q-learning algorithm without HP tuning. The fixed method has used the typical HP values: $\alpha = 0.1$ and $\gamma = 0.99$ [37], while the random method has used an arbitrary HP value at each episode. The random method has the same range that is used in the proposed method solution's bounds (Section 3.3). Furthermore, to ensure robustness and account for variability, each algorithm was executed 30 times. All the results were averaged over the second half of the episodes to ensure stability and provide a fair comparison, as the performance in the first half exhibited high deviation. However, the study results were obtained by conducting experiments on two OpenAI Gym control tasks: the Frozen Lake and Cart Pole, as explained in section 3.4. Subsequent sections will present the performance metrics and the detailed results.

4.1. Performance metrics

In this section, the performance of the proposed Q-FOX has been evaluated using the following performance metrics:

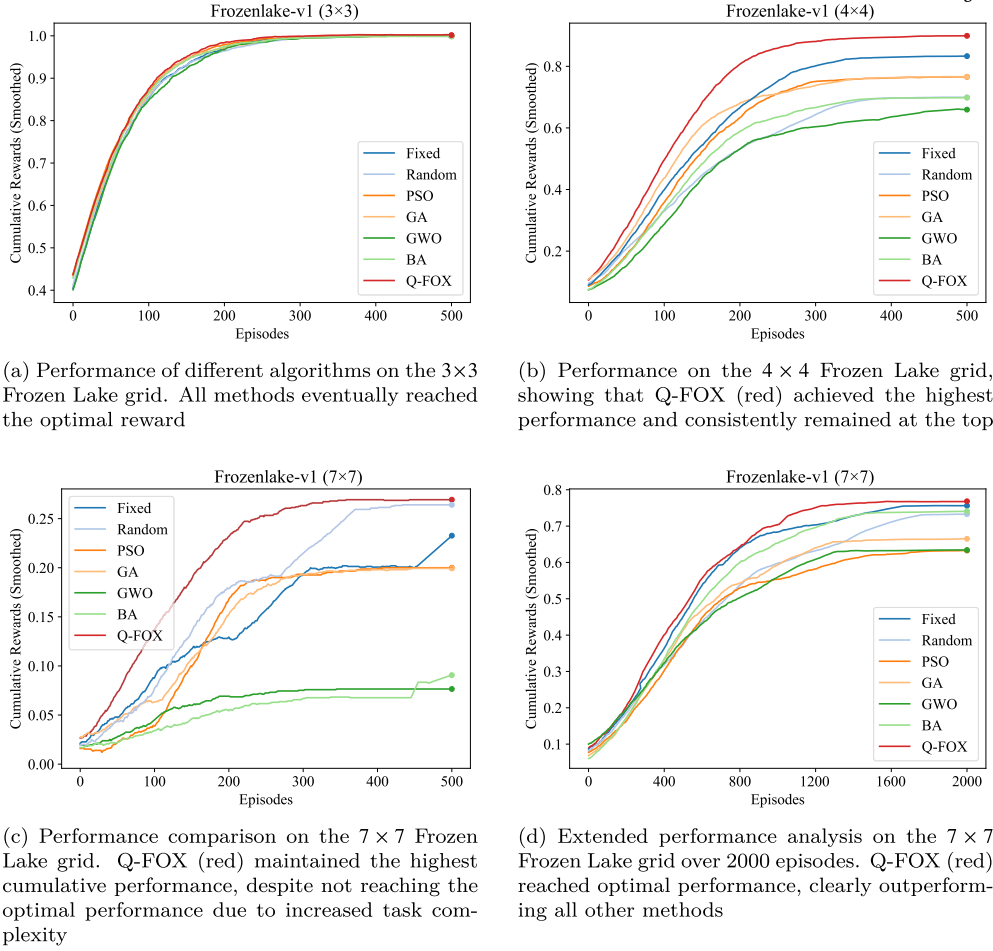


Fig. 5. Comparison of cumulative rewards across different grid sizes (3×3 , 4×4 , and 7×7) for the Frozen Lake control task.

Firstly, the *cumulative reward* represents the total reward accumulated over a series of episodes, providing insight into the algorithm's overall learning effectiveness and adaptability over time. This metric is essential for assessing the algorithm's ability to learn and improve its performance consistently [71]. It is calculated as:

$$R_{cumulative} = \sum_{t=1}^T r_t \quad (14)$$

Where r_t is the reward obtained at each time step t , and T is the total number of time steps. A higher cumulative reward indicates that the algorithm performs better in achieving its learning objectives over the entire task duration.

The *maximum reward* measures the highest reward achieved during a single episode, indicating the algorithm's peak performance potential. This metric is crucial for understanding the best-case scenario the algorithm can reach [72]:

$$R_{max} = \max(r_1, r_2, \dots, r_T) \quad (15)$$

This metric helps to identify how effectively the algorithm can reach optimal states or solutions within individual episodes, providing insight into its ability to exploit knowledge for maximum gain.

Lastly, the *processing time* indicates the total time to complete the learning process, reflecting the algorithm's efficiency. This metric is vital for real-world applications where time and computational resources are often limited [71,59]. It is measured in seconds and represented as:

$$T_{processing} = t_{end} - t_{start} \quad (16)$$

Where t_{start} is the time at the beginning of the learning process, and t_{end} is the time at the end of the learning process. This metric helps compare the computational efficiency of each algorithm, which is critical for determining their practicality and scalability in various applications.

4.2. Frozen Lake results

Fig. 5 illustrates the performance for three grid sizes: 3×3 , 4×4 , and 7×7 . Each subfigure includes seven plots representing the cumulative rewards for the fixed, random, PSO, GA, GWO, BA, and Q-FOX methods, coloured blue, light blue, orange, light orange, green, light green, and red, respectively. A simple moving average with a window size of 25 was applied to smooth the plots for better visualization. Firstly, Fig. 5a, which shows results for the 3×3 grid, demonstrates that all methods eventually reach the optimal reward, as this task is relatively simple. Notably, the red plot (Q-FOX) consistently appears above the others, indicating that the proposed method achieves faster convergence to optimal performance. Secondly, in Fig. 5b, which presents the 4×4 grid results, we observe that while not all methods reached optimal performance, Q-FOX achieved the highest performance and maintained its position at the top from start to finish, with no intersections from other plots. This result demonstrates its outstanding and consistent convergence speed. Similarly, Fig. 5c presents the results for the 7×7 grid. Here, the proposed Q-FOX method remained the top plot throughout, even though it did not reach the optimal performance due to the increased complexity of the task. This outcome demonstrates that the proposed method outperforms all other methods, even in more challenging task scenarios. Finally, Fig. 5d extends the analysis to the 7×7 grid with 2000 episodes because this task is complex and requires additional

Table 3

Performance results on Frozen Lake control task.

Grid Size	#Episodes	Algorithm	Cumulative reward	Max reward	STD	Processing Time (s)
3 × 3	500	Fixed	0.9993	1.0000	0.0000	8.9100
		Random	0.9996	1.0000	0.0000	8.4500
		PSO	0.9993	1.0000	0.0000	51.3700
		GA	1.0000	1.0000	0.0000	82.6000
		GWO	1.0000	1.0000	0.0000	58.2200
		BA	0.9993	1.0000	0.0000	78.4000
		Q-FOX	1.0000	1.0000	0.0000	57.7300
4 × 4	500	Fixed	0.8187	0.9000	0.0468	12.6724
		Random	0.6989	0.7000	0.0912	10.8847
		PSO	0.7651	0.7667	0.0648	81.0616
		GA	0.7658	0.7667	0.0699	125.8841
		GWO	0.7142	0.7000	0.0642	87.0245
		BA	0.7301	0.7000	0.0710	101.5010
		Q-FOX	0.8933	0.9214	0.0344	90.0239
7 × 7	500	Fixed	0.2222	0.2667	0.2610	8.1074
		Random	0.2662	0.2667	0.2404	6.8478
		PSO	0.2000	0.2000	0.2227	104.4235
		GA	0.1996	0.2000	0.2166	218.5172
		GWO	0.0667	0.0667	0.2829	101.9116
		BA	0.0653	0.0667	0.2618	102.2817
		Q-FOX	0.2671	0.3333	0.2014	235.2057
7 × 7	2000	Fixed	0.7322	0.7333	0.0866	12.2929
		Random	0.7322	0.7333	0.0741	10.9515
		PSO	0.6316	0.6333	0.1216	450.1791
		GA	0.6649	0.6667	0.0999	394.9435
		GWO	0.6338	0.6667	0.1168	259.6942
		BA	0.7662	0.8001	0.0723	383.4627
		Q-FOX	0.7951	0.8167	0.0649	225.9750
Q-FOX Avg. Results			0.7389	0.7679	0.1317	154.7337

processing time. The proposed Q-FOX method not only remained at the top but reached optimal performance, while none of the other methods achieved this outcome. Thus, Fig. 5 comprehensively demonstrates the superiority of Q-FOX across all scenarios, highlighting its effectiveness and consistent performance. However, Table 3 presents the performance results for various optimization algorithms applied to the Frozen Lake control task. The *Grid Size* column specifies the dimensions of the environment. *#Episodes* indicates the number of episodes each algorithm was tested, typically 500. For the 7×7 grid, the algorithm was tested across both 500 and 2000 episodes. *Algorithm* lists the tested methods, including fixed, random, PSO, GA, GWO, BA, and Q-FOX. *Cumulative reward* shows the total reward received through each episode as presented in Equation (14). Moreover, *Max reward* represents the highest reward obtained across algorithm runs as expressed in Equation (15). *STD* is the standard deviation that indicates the variability of cumulative rewards across the independent runs, providing insight into each algorithm's stability. Additionally, *Processing Time (s)* details the time taken to complete the learning process as explained in Equation (16). These metrics provide a comprehensive view of each algorithm's performance (see Section 4.1).

The proposed method demonstrated better results across four Frozen Lake control task experiments across all other algorithms, achieving outcomes within the expected range. Specifically, it reached near the maximum reward in most cases. Achieving cumulative reward of 1, 0.8933, 0.2671, 0.7951 for 500 episodes in 3×3 , 4×4 , 7×7 , and 7×7 (2000 episodes) grids, respectively.

In the last row of Table 3, the results of the proposed method have been averaged across all grid sizes, indicating a cumulative reward of 0.7389 and a maximum reward of 0.7679. The following section will list the results of the Cart Pole control task.

4.3. Cart Pole results

The performance results illustrated in Fig. 6 of the Cart Pole task, including seven plots representing the fixed method, random method,

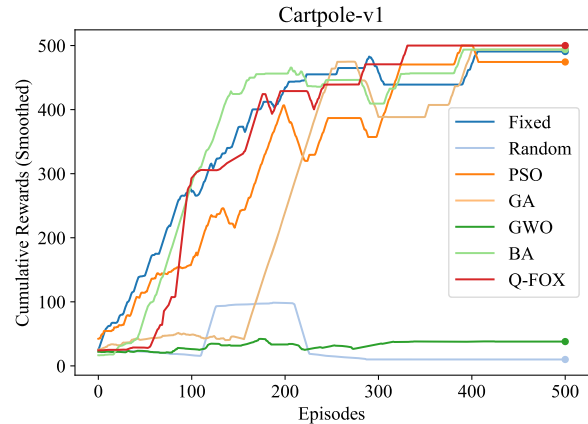


Fig. 6. Performance comparison on the Cart Pole task for seven methods: fixed, random, PSO, GA, GWO, BA, and Q-FOX (coloured blue to red). Q-FOX ultimately achieved the highest reward (500), demonstrating superior performance despite the task's complexity.

PSO, GA, GWO, BA, and Q-FOX coloured in blue, light blue, orange, light orange, green, light green, and red, respectively. With closer inspection, it is evident that the proposed method initially overlaps with other methods during the early stages of learning, following the fixed and the BA. However, as learning progresses, the proposed method reaches the highest possible reward (500). Furthermore, Table 4 indicate that the proposed method outperforms other methods. Q-FOX's cumulative reward was 500, the fixed method was 483.98, the random method was 10.0133, the PSO was 481.5481, the GA was 478.9467, the GWO was 37.96, and the BA was 486.325. Additionally, the fluctuations observed in Fig. 6 can be attributed to the complexity of the problem environment, which presents more challenges than the Frozen Lake task. Despite this increased complexity, the proposed method demonstrates its superiority by achieving the highest level of performance.

Table 4

Performance results on Cart Pole control task.

#Episodes	Algorithm	Cumulative reward	Max reward	STD	Processing Time (s)
500	Fixed	483.9800	500.0000	5.7088	589.6594
	Random	10.0133	12.0000	17.7940	594.2946
	PSO	481.5481	495.7291	5.2130	7,619.3425
	GA	478.9467	500.0000	5.4617	12,596.0600
	GWO	37.9600	63.0000	8.3456	7,826.9932
	BA	486.3250	498.3540	4.6296	8,237.0043
	Q-FOX	500.0000	500.0000	3.6112	9,609.7589

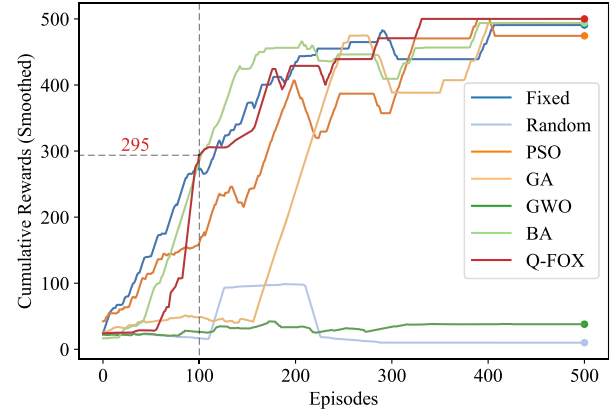
Tables 3 and 4 present the results of the proposed Q-FOX and other optimization algorithms, including conventional Q-learning (fixed and random methods), across both the Frozen Lake and Cart Pole control tasks. For the Frozen Lake, results across different grid sizes (3×3 , 4×4 , and 7×7) show that Q-FOX consistently achieves the highest average performance, outperforming conventional Q-learning by 8.15%. Specifically, Q-FOX's average performance was 0.7534, exceeding that of conventional Q-learning of 0.6918, demonstrating its superior HP tuning. In the Cart Pole, Q-FOX reached the maximum possible average performance of 500, outperforming conventional Q-learning by 49.73% (Q-FOX: 500 vs conventional Q-learning: 251.4983), highlighting its enhanced learning capabilities. Although Q-FOX is not always the fastest in computational efficiency, the trade-off significantly improves reward metrics. Finally, Q-FOX consistently achieves higher cumulative and maximum rewards than conventional Q-learning, validating its effectiveness in adaptive HP tuning that improved the conventional Q-learning.

The standard deviation (STD) of cumulative rewards across 30 independent runs was analyzed to evaluate the robustness and stability of the Q-FOX algorithm compared to other methods. In the Frozen Lake control task with a 4×4 grid size, Q-FOX demonstrated superior performance with a lower STD of 0.0344, outperforming other algorithms, including fixed with 0.0468, random with 0.0912, PSO with 0.0648, GA with 0.0699, GWO with 0.0642, and BA with 0.0710. This result indicates that Q-FOX was 47.48% more stable than other methods. Similarly, in the 7×7 grid, the STD for Q-FOX was 0.2014, which was lower than fixed with 0.2610, random with 0.2404, PSO with 0.2227, GA with 0.2166, GWO with 0.2829, and BA with 0.2618, showing that Q-FOX maintained more consistent behaviour and was 17.91% more stable. When evaluated over 2000 episodes in the 7×7 grid, Q-FOX had an STD of 0.0649, outperforming fixed with 0.0866, random with 0.0741, PSO with 0.1216, GA with 0.0999, GWO with 0.1168, and BA with 0.0723, indicating an overall stability advantage of 28.97%. The Frozen Lake control task thus confirmed Q-FOX's consistent superiority, with an average stability improvement of 31.45% across all cases. Furthermore, in the Cart Pole control task, Q-FOX achieved an STD of 3.6112, significantly outperforming fixed with 5.7088, random with 17.7940, PSO with 5.2130, GA with 5.4617, GWO with 8.3456, and BA with 4.6296, resulting in a 43.30% improvement in stability. These findings demonstrate that Q-FOX achieved an average stability enhancement of 37.38% across both tasks, proving its robustness in providing consistent performance.

However, the following section thoroughly compares Q-FOX with other optimization algorithms and conventional Q-learning. This analysis aims to understand better why the proposed method excelled in solving the Frozen Lake and Cart Pole control tasks relative to the other algorithms, discussing the factors contributing to its superiority and any associated limitations.

5. Discussion

Despite advancements in RL, issues such as getting stuck in local minima and inefficiencies in HP tuning methods still need to be solved.

**Fig. 7.** Cumulative reward of Cart Pole control task at episode 100.

Although optimization algorithms like PSO, GWO, GA, and BA have progressed in RL algorithms, they face limitations such as long convergence times and suboptimal reward. The proposed Q-FOX addresses these limitations by introducing an efficient adaptive HP tuning method and a robust objective function. This method effectively maximizes reward and avoids local minima in control tasks like Cart Pole and Frozen Lake, outperforming the performance of existing optimization algorithms.

This study found that cumulative reward increased and reached optimal levels in most cases as shown in Table 3, the highest possible cumulative reward was achieved in the Frozen Lake control task for grid sizes of 3×3 and 4×4 . However, for the 7×7 grid, the task's complexity prevented the optimal solution within 500 episodes. The proposed method reached the optimal solution after extending the experiment to 2000 episodes. Additionally, Table 4 demonstrates that the proposed method achieved the maximum cumulative reward in the Cart Pole control task.

Unlike the study [37], which used the average success ratio to evaluate the Frozen Lake control task, our study employed average cumulative reward. We calculated the success ratio from our results to the average success ratio to ensure a fair comparison. The success ratio is the number of times the agent reached the goal (cumulative reward) divided by the total steps taken. Their study used a 4×4 grid with 1500 episodes, achieving a best average success ratio of approximately 0.0244. In contrast, we used 500 episodes and reached an average success ratio of 0.045 in the same environment, with an average of 20 steps. Additionally, the study [38] utilized a 4×4 grid with three environments differing in the number and location of holes, running 10,000 episodes for each task. Given the simplicity of the environment, we found this number of episodes to be excessive. However, we focused on episode 500 for comparison purposes, as achieving optimal performance within this limited number of episodes is more challenging. Our average cumulative reward for this task was 0.8933, with a maximum of 0.9214, whereas their maximum average at episode 500 was approximately 0.2. This suggests that our proposed method demonstrates faster convergence to optimal reward with fewer episodes compared to [37,38].

Although the study [39] achieved good results in the Cart Pole control task over 100 episodes, it exhibited low cumulative reward in addition to some deviation in convergence, with a maximum reward of approximately 88 and an average reward of 26. In contrast, our results for the same task at episode 100 showed a better reward of roughly 295, as illustrated in Fig. 7.

The results in Tables 3 and 4 indicate that the Q-FOX outperformed other optimization algorithms, including PSO, GA, GWO, and BA [26,23–25], as well as the fixed and random methods, in terms of both average cumulative reward and maximum reward. Specifically, Q-FOX surpassed the fixed, random, PSO, and BA in the Frozen Lake

control task with a 3×3 grid. It also performed better on the 4×4 and 7×7 grids than all other algorithms.

In Fig. 5d, although the Q-FOX did not reach the optimal cumulative reward, it demonstrated significantly faster convergence to the optimal reward than other algorithms. Similarly, Q-FOX outperformed all other algorithms in average cumulative and maximum reward metrics in the Cart Pole control task.

Q-FOX's superior performance can be attributed to its adaptive HP tuning method, which allows it to dynamically adjust learning rates and discount factors (α , γ) that also influence the exploration strategies based on the task's complexity. This adaptability enables Q-FOX to efficiently navigate complex state spaces, such as the 7×7 grid in the Frozen Lake control task, thereby avoiding local minima more effectively than conventional Q-learning and optimization algorithms like PSO, GA, GWO, and BA. However, this adaptability comes with a trade-off in processing time, as Q-FOX's iterative optimization process requires more computational effort. While this makes Q-FOX effective in maximizing reward, it also indicates that the method has longer learning times than conventional Q-learning (fixed and random), making it less suitable for real-time applications where quick decision-making is essential.

However, the processing time metric requires a bit consideration due to noticeable variations. The fixed and random methods exhibited lower processing times because they rely only on Q-learning without any optimization or tuning. In contrast, the optimization-based methods involve an additional optimization process at each step of the Q-learning. For instance, if the fixed method performs 500 episodes with 200 steps, the total number of iterations will be $500 \times 200 = 100K$. On the other hand, tuning-based methods, such as Q-FOX and the others, execute 50 optimization epochs with 20 agents at each step, resulting in $500 \times 200 \times 50 \times 20 = 100M$ iterations. Given this significant difference, the fixed and random methods are excluded from further analysis of processing time, with the focus placed on the optimization algorithm for a more meaningful comparison.

Discussing why the proposed method takes more processing time than the fixed and random methods is crucial. Q-FOX consistently achieved higher cumulative rewards in the Frozen Lake and Cart Pole tasks, outperforming other algorithms, including the fixed and random method, especially in complex scenarios with larger grid sizes. Unlike the fixed and random method's static values, its adaptive HP tuning allows efficient exploration and exploitation during learning. Q-FOX has also demonstrated its ability to generalize across different control tasks with reliable results, which is crucial for RL applications where optimal HP may vary. In larger-scale tasks, such as the 7×7 grid, Q-FOX achieved a cumulative reward of 0.2671, surpassing the fixed method's 0.2222 and the random method's 0.2662 over 500 episodes, highlighting its effectiveness in challenging environments.

In the 3×3 grid size Frozen Lake control task, the processing time of the proposed method was 57.73 seconds, which was lower than that of all other algorithms except PSO. For the 4×4 grid, the proposed method it was lower than only GA and BA. However, in the 7×7 grid size, it was the worst among the tested algorithms. Despite this, in the 7×7 grid with 2000 episodes, the proposed method achieved the best performance with the lowest processing time of 225.975 seconds. In the Cart Pole control task, the proposed method's processing time was the second worst. These variations in processing time underscore the inherent variability of the optimization process. Therefore, this study emphasized performance in terms of reward maximization rather than processing time.

Despite the success of Q-FOX, it has encountered some challenges. One limitation is the extended learning time compared to conventional Q-learning, which can be a notable limitation for real-time applications. Additionally, selecting the optimal parameters during the learning process requires a simulation environment. Implementing this learning process in a real-world setting causes difficulties, such as excessive energy consumption by the agent. Consequently, the proposed method will

remain restricted to simulation environments until further research addresses these limitations. Our study reveals challenges with long learning time and reliance only on simulations. Future research could optimize the learning process by improving this proposed method based on transfer learning or utilizing parallel processing, improving performance and demonstrating versatility.

6. Conclusion

To conclude, this paper introduces a novel reinforcement learning tuning method called Q-learning-FOX (Q-FOX), which adaptively tunes hyperparameters in Q-learning. An essential contribution is replacing the static hyperparameter in the Q-learning and utilizing an adaptive method based on the FOX optimization algorithm. Moreover, a new objective function has been proposed that combines average Q -values and learning error into a unified fitness function. This approach is designed to optimize hyperparameters more effectively by selecting those that maximize the overall reward. The experimental results from OpenAI Gym environments, specifically the Frozen Lake and Cart Pole control tasks, demonstrate that Q-FOX outperforms conventional Q-learning, which uses established optimization algorithms regarding convergence speed and maximum reward. Q-FOX achieved up to a 12.59% higher cumulative reward in the Frozen Lake control task and 34.16% higher cumulative reward in the Cart Pole control task compared to established optimization algorithms such as PSO, GA, GWO, and BA. This method effectively addresses the limitations of manual or random hyperparameter tuning, offering a more efficient alternative. However, it is crucial to note that using Q-FOX in real-world scenarios may be time-consuming because optimization requires an iterative process. Future work will study improving the proposed method by using transfer learning or parallel processing to enhance the efficiency of Q-FOX in real-world applications.

CRedit authorship contribution statement

Mahmood A. Jumaah: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **Yossra H. Ali:** Writing – review & editing, Validation, Supervision, Project administration, Investigation. **Tarik A. Rashid:** Writing – review & editing, Validation, Supervision, Project administration, Investigation, Conceptualization.

Funding

The authors received no specific funding for this study.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

References

- [1] D. Huergo, L. Alonso, S. Joshi, A. Juanicotena, G. Rubio, E. Ferrer, A reinforcement learning strategy to automate and accelerate h/p-multigrid solvers, *Results Eng.* 24 (2024) 102949, <https://doi.org/10.1016/j.rineng.2024.102949>, <https://www.sciencedirect.com/science/article/pii/S2590123024012040>.
- [2] F.-M. Luo, T. Xu, H. Lai, X.-H. Chen, W. Zhang, Y. Yu, A survey on model-based reinforcement learning, *Sci. China Inf. Sci.* 67 (2) (Jan. 2024), <https://doi.org/10.1007/s11432-022-3696-5>.

- [3] J. Zhou, S. Xue, Y. Xue, Y. Liao, J. Liu, W. Zhao, A novel energy management strategy of hybrid electric vehicle via an improved td3 deep reinforcement learning, *Energy* 224 (2021) 120118, <https://doi.org/10.1016/j.energy.2021.120118>.
- [4] P. Pontrandolfo, A. Gosavi, O.G. Okogbaa, T.K. Das, Global supply chain management: a reinforcement learning approach, *Int. J. Prod. Res.* 40 (6) (2002) 1299–1317, <https://doi.org/10.1080/00207540110118640>.
- [5] M. Zuccotto, A. Castellini, D.L. Torre, L. Mola, A. Farinelli, Reinforcement learning applications in environmental sustainability: a review, *Artif. Intell. Rev.* 57 (4) (Mar. 2024), <https://doi.org/10.1007/s10462-024-10706-5>.
- [6] A. Neufeld, J. Sester, Robust Q -learning algorithm for Markov decision processes under Wasserstein uncertainty, *Automatica* 168 (2024) 111825, <https://doi.org/10.1016/j.automatica.2024.111825>.
- [7] A.T. Khudhair, A.T. Maolood, E.K. Gbashi, Symmetry analysis in construction two dynamic lightweight s-boxes based on the 2d Tinkerbell map and the 2d Duffing map, *Symmetry* 16 (7) (2024) 872, <https://doi.org/10.3390/sym16070872>.
- [8] A.T. Khudhair, A.T. Maolood, E.K. Gbashi, Symmetric keys for lightweight encryption algorithms using a pre-trained vgg16 model, *Telecom* 5 (3) (2024) 892–906, <https://doi.org/10.3390/telecom5030444>.
- [9] X. Ni, W. Hu, Q. Fan, Y. Cui, C. Qi, A q -learning based multi-strategy integrated artificial bee colony algorithm with application in unmanned vehicle path planning, *Expert Syst. Appl.* 236 (2024) 121303, <https://doi.org/10.1016/j.eswa.2023.121303>.
- [10] N. Ali, A. Wahid, R. Shaw, K. Mason, A reinforcement learning approach to dairy farm battery management using q learning, *J. Energy Storage* 93 (2024) 112031, <https://doi.org/10.1016/j.est.2024.112031>.
- [11] S. Jabber, S. Hashem, S. Jafer, Analytical and comparative study for optimization problems, *Iraqi J. Comput. Commun. Control Syst. Eng.* 23 (4) (2023) 46–57, <https://doi.org/10.33103/uot.ijccce.23.4.5>, <https://ijccce.uotechnology.edu.iq/article/178584.html>.
- [12] S. Hanifi, A. Cammarono, H. Zare-Behtash, Advanced hyperparameter optimization of deep learning models for wind power prediction, *Renew. Energy* 221 (2024) 119700, <https://doi.org/10.1016/j.renene.2023.119700>.
- [13] B. Bischl, M. Binder, M. Lang, T. Pielok, J. Richter, S. Coors, J. Thomas, T. Ullmann, M. Becker, A. Boulesteix, D. Deng, M. Lindauer, Hyperparameter optimization: foundations, algorithms, best practices, and open challenges, *WIREs Data Min. Knowl. Discov.* 13 (2) (Jan. 2023), <https://doi.org/10.1002/widm.1484>.
- [14] P. Ribino, C. Di Napoli, G. Paragliola, L. Serino, Hyper-parameter optimization through reinforcement learning for survival prediction of patients with heart failure, *Proc. Comput. Sci.* 239 (2024) 1754–1761, <https://doi.org/10.1016/j.procs.2024.06.354>.
- [15] M. Turki Almousa, M.R. Goma, M. Ghasemi, M. Louzazni, Gradient-based optimization for parameter identification of lithium-ion battery model for electric vehicles, *Results Eng.* 24 (2024) 102845, <https://doi.org/10.1016/j.rineng.2024.102845>.
- [16] S. Vijayakumar, N. Sudhakar, Design and implementation of golden eagle optimized cascaded pi and lqr controller for pfc sepic converter in ev charging, *Results Eng.* 24 (2024) 102942, <https://doi.org/10.1016/j.rineng.2024.102942>.
- [17] A. Muqet, A. Israr, M.H. Zafar, M. Mansoor, N. Akhtar, A novel optimization algorithm based pid controller design for real-time optimization of cutting depth and surface roughness in finish hard turning processes, *Results Eng.* 18 (2023) 101142, <https://doi.org/10.1016/j.rineng.2023.101142>.
- [18] H. Abdulkarim, T.A. Rashid, In search of excellence: shoa as a competitive shrike optimization algorithm for multimodal problems, <https://doi.org/10.36227/techrxiv.171169350.05333436/v1>, Mar. 2024.
- [19] M. Baake, F. Gähler, J. Mazáč, On the Fibonacci tiling and its modern ramifications, *Isr. J. Chem.* (Apr. 2024), <https://doi.org/10.1002/ijch.202300155>.
- [20] S. Abdulhameed, T.A. Rashid, Child drawing development optimization algorithm based on child's cognitive development, *Arab. J. Sci. Eng.* 47 (2) (2021) 1337–1351, <https://doi.org/10.1007/s13369-021-05928-6>.
- [21] D.N. Hama Rashid, T.A. Rashid, S. Mirjalili, Ana: ant nesting algorithm for optimizing real-world problems, *Mathematics* 9 (23) (2021) 3111, <https://doi.org/10.3390/math9233111>.
- [22] J.M. Abdullah, T. Ahmed, Fitness dependent optimizer: inspired by the bee swarming reproductive process, *IEEE Access* 7 (2019) 43473–43486, <https://doi.org/10.1109/access.2019.2907012>.
- [23] F. Cardenoso Fernandez, W. Caarls, Parameters tuning and optimization for reinforcement learning algorithms using evolutionary computing, in: 2018 International Conference on Information Systems and Computer Science (INCISOS), 2018, pp. 301–305.
- [24] D. Tapia, B. Crawford, R. Soto, F. Cisternas-Caneo, J. Lemus-Romani, M. Castillo, J. García, W. Palma, F. Paredes, S. Misra, A Q -Learning Hyperheuristic Binarization Framework to Balance Exploration and Exploitation, *Springer International Publishing*, 2020, pp. 14–28.
- [25] Y.A.Z.A. Fajri, W. Wiharto, E. Suryani, Hybrid model feature selection with the bee swarm optimization method and q -learning on the diagnosis of coronary heart disease, *Information* 14 (1) (2022) 15, <https://doi.org/10.3390/info14010015>.
- [26] L. Tan, H. Zhang, Y. Liu, T. Yuan, X. Jiang, Z. Shang, An adaptive q -learning based particle swarm optimization for multi-uav path planning, *Soft Comput.* 28 (13–14) (2024) 7931–7946, <https://doi.org/10.1007/s00500-024-09691-2>.
- [27] H. Mohammed, T. Rashid, Fox: a fox-inspired optimization algorithm, *Appl. Intell.* 53 (1) (2022) 1030–1050, <https://doi.org/10.1007/s10489-022-03533-0>.
- [28] S. Moyya, P. Thejasree, B. Cherian Abraham, G.S. Mangalathu, Design and analysis of single and multi-layer pressure vessel, in: *Materials Today: Proceedings*, Jul. 2023.
- [29] N. Singh, T. Chakrabarti, P. Chakrabarti, M. Margala, A. Gupta, S.P. Praveen, S.B. Krishnan, B. Unhelkar, Novel heuristic optimization technique to solve economic load dispatch and economic emission load dispatch problems, *Electronics* 12 (13) (2023) 2921, <https://doi.org/10.3390/electronics12132921>.
- [30] S.N. Agos Jawaddi, A. Ismail, Integrating openai gym and cloudsim plus: a simulation environment for drl agent training in energy-driven cloud scaling, *Simul. Model. Pract. Theory* 130 (2024) 102858, <https://doi.org/10.1016/j.simpact.2023.102858>.
- [31] X. Ding, M. Zheng, X. Zheng, The application of genetic algorithm in land use optimization research: a review, *Land* 10 (5) (2021) 526, <https://doi.org/10.3390/land10050526>.
- [32] S. Mirjalili, S.M. Mirjalili, A. Lewis, Grey wolf optimizer, *Adv. Eng. Softw.* 69 (2014) 46–61, <https://doi.org/10.1016/j.advengsoft.2013.12.007>.
- [33] D. Pham, A. Ghanbarzadeh, E. Koç, S. Otri, S. Rahim, M. Zaidi, *The Bees Algorithm — A Novel Tool for Complex Optimisation Problems*, Elsevier, 2006, pp. 454–459.
- [34] D. Wang, D. Tan, L. Liu, Particle swarm optimization algorithm: an overview, *Soft Comput.* 22 (2) (2017) 387–408, <https://doi.org/10.1007/s00500-016-2474-6>.
- [35] A.L.C. Ottoni, E.G. Nepomuceno, M.S. de Oliveira, D.C.R. de Oliveira, Tuning of reinforcement learning parameters applied to sop using the Scott–Knott method, *Soft Comput.* 24 (6) (2019) 4441–4453, <https://doi.org/10.1007/s00500-019-04206-w>.
- [36] T.L. Paine, C. Paduraru, A. Michi, C. Gulcehre, K. Zolna, A. Novikov, Z. Wang, N. de Freitas, Hyperparameter selection for offline reinforcement learning, <https://doi.org/10.48550/ARXIV.2007.09055>, <https://arxiv.org/abs/2007.09055>, 2020.
- [37] A. Gupta, P.P. Roy, V. Dutt, Evaluation of instance-based learning and q -learning algorithms in dynamic environments, *IEEE Access* 9 (2021) 138775–138790, <https://doi.org/10.1109/access.2021.3117855>.
- [38] H.Y. Mak, F.X. Fan, L.A. Lanzendorf, C. Tan, W.T. Ooi, R. Wattenhofer, Caesar: enhancing federated rl in heterogeneous mdps through convergence-aware sampling with screening, <https://doi.org/10.48550/ARXIV.2403.20156>, <https://arxiv.org/abs/2403.20156>, 2024.
- [39] S. Mishra, A. Arora, Balancing a cart pole using reinforcement learning in openai gym environment, *SSRN Electron. J.* (2024), <https://doi.org/10.2139/ssrn.4768234>.
- [40] T.M. Moerland, J. Broekens, A. Plaat, C.M. Jonker, Model-based reinforcement learning: a survey, *Found. Trends Mach. Learn.* 16 (1) (2023) 1–118, <https://doi.org/10.1561/22000000086>.
- [41] A. Kopacz, L. Csató, C. Chira, Evaluating cooperative-competitive dynamics with deep q -learning, *Neurocomputing* 550 (2023) 126507, <https://doi.org/10.1016/j.neucom.2023.126507>.
- [42] Z. Ning, L. Xie, A survey on multi-agent reinforcement learning and its application, *J. Autom. Intell.* 3 (2) (2024) 73–91, <https://doi.org/10.1016/j.jai.2024.02.003>.
- [43] S. Jayakumar, S. Nandakumar, Distributed resource optimisation using the q -learning algorithm, in device-to-device communication: a reinforcement learning paradigm, *Results Eng.* 23 (2024) 102462, <https://doi.org/10.1016/j.rineng.2024.102462>.
- [44] R.S. Sutton, A.G. Barto, *Reinforcement Learning: An Introduction*, A Bradford Book, Cambridge, MA, USA, 2018.
- [45] S. Wei, Y. Bao, H. Li, Optimal policy for structure maintenance: a deep reinforcement learning framework, *Struct. Saf.* 83 (2020) 101906, <https://doi.org/10.1016/j.strusafe.2019.101906>.
- [46] F. Chang, T. Chen, W. Su, Q. Alsafasfeh, Charging control of an electric vehicle battery based on reinforcement learning, in: 2019 10th International Renewable Energy Congress (IREC), 2019, pp. 1–63, <https://ieeexplore.ieee.org/abstract/document/8754518>.
- [47] M. Imani, H.R. Arabnia, Hyperparameter optimization and combined data sampling techniques in machine learning for customer churn prediction: a comparative analysis, *Technologies* 11 (6) (2023) 167, <https://doi.org/10.3390/technologies11060167>.
- [48] G. Békési, L. Barancsik, B. Hartmann, Deep neural network based distribution system state estimation using hyperparameter optimization, *Results Eng.* 24 (2024) 102908, <https://doi.org/10.1016/j.rineng.2024.102908>.
- [49] Z.K. Abdalrhah, A.M. Al-Bakry, A.K. Farhan, Cnn hyper-parameter optimizer based on evolutionary selection and Gow approach for crimes tweet detection, in: 2023 16th International Conference on Developments in eSystems Engineering (DeSE), IEEE, 2023.
- [50] P. Ladosz, L. Weng, M. Kim, H. Oh, Exploration in deep reinforcement learning: a survey, *Inf. Fusion* 85 (2022) 1–22, <https://doi.org/10.1016/j.inffus.2022.03.003>.
- [51] Q. Liu, Z. Liu, B. Xiong, W. Xu, Y. Liu, Deep reinforcement learning-based safe interaction for industrial human-robot collaboration using intrinsic reward function, *Adv. Eng. Inform.* 49 (2021) 101360, <https://doi.org/10.1016/j.aei.2021.101360>.
- [52] Y. Liu, B. Cao, H. Li, Improving ant colony optimization algorithm with epsilon greedy and Levy flight, *Complex Intell. Syst.* 7 (4) (2020) 1711–1722, <https://doi.org/10.1007/s40747-020-00138-3>.
- [53] T. Yang, S. Zhang, C. Li, A multi-objective hyper-heuristic algorithm based on adaptive epsilon-greedy selection, *Complex Intell. Syst.* 7 (2) (2021) 765–780, <https://doi.org/10.1007/s40747-020-00230-8>.
- [54] X. Qi, B. Xu, Hyperparameter optimization of neural networks based on q -learning, *Signal Image Video Process.* 17 (4) (2022) 1669–1676, <https://doi.org/10.1007/s11760-022-02377-y>.
- [55] V. Shalamov, V. Efimova, S. Muravyov, A. Filchenkov, Reinforcement-based method for simultaneous clustering algorithm selection and its hyperparameters optimization, *Proc. Comput. Sci.* 136 (2018) 144–153, <https://doi.org/10.1016/j.procs.2018.08.247>.

- [56] R. Sauerheber, Light and sound speed mechanisms, relative velocities, simultaneity, and special relativity, SSRN Electron. J. (2024), <https://doi.org/10.2139/ssrn.4780574>.
- [57] Y. Hussain Ali, V. Sabu Chooralil, K. Balasubramanian, R.R. Manyam, S. Kidambi Raju, A.T. Sadiq, A.K. Farhan, Optimization system based on convolutional neural network and Internet of medical things for early diagnosis of lung cancer, Bioengineering 10 (3) (2023) 320, <https://doi.org/10.3390/bioengineering10030320>.
- [58] H. Mutar, M. Jawad, Analytical study for optimization techniques to prolong wsns life, Iraqi J. Comput. Commun. Control Syst. Eng. 23 (2) (2023) 13–23, <https://doi.org/10.33103/uot.ijccce.23.2.2>, https://ijccce.uotechnology.edu.iq/article_178569.html.
- [59] M.A. Jumaah, Y.H. Ali, T.A. Rashid, S. Vimal, FOXANN: a method for boosting neural network performance, J. Soft Comput. Comput. Appl. 1 (Jun. 2024), <https://doi.org/10.48550/ARXIV.2407.03369>.
- [60] M.A. Jumaah, A.I. Shihab, A.A. Farhan, Epileptic seizures detection using det-ii and knn classifier in long-term eeg signals, Iraqi J. Sci. (2020) 2687–2694, <https://doi.org/10.24996/ijis.2020.61.10.26>.
- [61] Z. Waheed, A. Humaidi, Whale optimization algorithm enhances the performance of knee-exoskeleton system controlled by smc, Iraqi J. Comput. Commun. Control Syst. Eng. 23 (2) (2023) 125–135, <https://doi.org/10.33103/uot.ijccce.23.2.10>, https://ijccce.uotechnology.edu.iq/article_178337.html.
- [62] M.H. Ismael, A.T. Maolood, Withdrawn: developing modern system in healthcare to detect covid 19 based on Internet of things, in: Materials Today: Proceedings, Jun. 2021.
- [63] M.A. Jumaah, Y.H. Ali, T.A. Rashid, Q-learning-fox (qfox), <https://github.com/mwdx93/qfox>, 2024. (Accessed 29 September 2024).
- [64] H. Alavizadeh, H. Alavizadeh, J. Jang-Jaccard, Deep q-learning based reinforcement learning approach for network intrusion detection, Computers 11 (3) (2022) 41, <https://doi.org/10.3390/computers11030041>.
- [65] M.L.H.D. Lemos, R.E.S. Vieira, A.R. Tavares, L.S. Marcolino, L. Chaimowicz, Enhancing deep reinforcement learning for scale flexibility in real-time strategy games, Entertain. Comput. 52 (2025) 100843, <https://doi.org/10.1016/j.entcom.2024.100843>.
- [66] A.G. Barto, R.S. Sutton, C.W. Anderson, Neuronlike adaptive elements that can solve difficult learning control problems, IEEE Trans. Syst. Man Cybern. SMC-13 (5) (1983) 834–846, <https://doi.org/10.1109/TSMC.1983.6313077>, <https://ieeexplore.ieee.org/document/6313077>.
- [67] S. Mishra, A. Arora, A Huber reward function-driven deep reinforcement learning solution for cart-pole balancing problem, Neural Comput. Appl. 35 (23) (2022) 16705–16722, <https://doi.org/10.1007/s00521-022-07606-6>.
- [68] M.J. Mahmoodabadi, M. Andalib Sahnehsaraei, Parametric uncertainty handling of under-actuated nonlinear systems using an online optimal input–output feedback linearization controller, Syst. Sci. Control Eng. 9 (1) (2021) 209–218, <https://doi.org/10.1080/21642583.2021.1891993>.
- [69] W. Pilar von Pilchau, Text/conference paper, https://doi.org/10.18420/INF2019_WS53, <http://dl.gi.de/handle/20.500.12116/25089>, 2019.
- [70] Z. Zhang, S.S. Du, X. Ji, Nearly minimax optimal reward-free reinforcement learning, <https://doi.org/10.48550/ARXIV.2010.05901>, <https://arxiv.org/abs/2010.05901>, 2020.
- [71] P. Vamplew, R. Dazeley, A. Berry, R. Issabekov, E. Dekker, Empirical evaluation methods for multiobjective reinforcement learning algorithms, Mach. Learn. 84 (1–2) (2010) 51–80, <https://doi.org/10.1007/s10994-010-5232-5>.
- [72] G. Dulac-Arnold, N. Levine, D.J. Mankowitz, J. Li, C. Paduraru, S. Gowal, T. Hester, Challenges of real-world reinforcement learning: definitions, benchmarks and analysis, Mach. Learn. 110 (9) (2021) 2419–2468, <https://doi.org/10.1007/s10994-021-05961-4>.